

2.2 - What Is Algorithm Analysis?

It is very common for beginning computer science students to compare their programs with one another. You may also have noticed that it is common for computer programs to look very similar, especially the simple ones. An interesting question often arises. When two programs solve the same problem but look different, is one program better than the other?

In order to answer this question, we need to remember that there is an important difference between a program and the underlying algorithm that the program is representing. As we stated in Chapter 1, an algorithm is a generic, step-by-step list of instructions for solving a problem. It is a method for solving any instance of the problem so that given a particular input, the algorithm produces the desired result. A program, on the other hand, is an algorithm that has been encoded into some programming language. There may be many programs for the same algorithm, depending on the programmer and the programming language being used.

To explore this difference further, consider the function shown in [Listing 2.2.1](#). This function solves a familiar problem, computing the sum of the first n integers. The algorithm uses the idea of an accumulator variable that is initialized to 0. The solution then iterates through the n integers, adding each to the accumulator.

Listing 2.2.1.

```
public class FindSum {
    public static long sumOfN(int n) {
        long theSum = 0;
        for (int i = 1; i <= n; i++) {
            theSum = theSum + i;
        }
        return theSum;
    }

    public static void main(String[] args) {
        System.out.println(sumOfN(10));
    }
}
```

► Aside

Now look at the method in [Listing 2.2.2](#). At first glance it may look strange, but upon further inspection you can see that this function is essentially doing the same thing as the previous one. The reason this is not obvious is poor coding. We did not use good identifier names to assist with readability, and we used an extra assignment statement that was not really necessary during the accumulation step.

Listing 2.2.2.

```
public class FindSum2 {  
    public static long foo(int tom) {  
        long fred = 0;  
        for (int nancy = 1; nancy <= tom; nancy++) {  
            long joanne = nancy;  
            fred = fred + joanne;  
        }  
        return fred;  
    }  
  
    public static void main(String[] args) {  
        System.out.println(foo(10));  
    }  
}
```

The question we raised earlier asked whether one method is better than another. The answer depends on your criteria. The method `sumOfN` is certainly better than the method `foo` if you are concerned with readability. In fact, you have probably seen many examples of this in your introductory programming course since one of the goals there is to help you write programs that are easy to read and easy to understand. In this course, however, we are also interested in characterizing the algorithm itself. (We certainly hope that you will continue to strive to write readable, understandable code.)

Algorithm analysis is concerned with comparing algorithms based upon the amount of computing resources that each algorithm uses. We want to be able to consider two algorithms and say that one is better than the other because it is more efficient in its use of those resources or perhaps because it simply uses fewer. From this perspective, the two methods above seem very similar. They both use essentially the same algorithm to solve the summation problem.

At this point, it is important to think more about what we really mean by computing resources. There are two different ways to look at this. One way is to

consider the amount of space or memory an algorithm requires to solve the problem. The amount of space required by a problem solution is typically dictated by the problem instance itself. Every so often, however, there are algorithms that have very specific space requirements, and in those cases we will be very careful to explain the variations.

As an alternative to space requirements, we can analyze and compare algorithms based on the amount of time they require to execute. This measure is sometimes referred to as the *execution time* or *running time* of the algorithm. One way we can measure the execution time for the function `sumOfN` is to do a **benchmark analysis**. This means that we will track the actual time required for the program to compute its result. In Java, we can benchmark a function by noting the starting time and ending time within the system we are using. In the `System` module there is a method called `nanotime` that will return the amount of time that the Java virtual machine has been running, in nanoseconds. By calling this function twice, at the beginning and at the end, and then computing the difference, we can get the number of seconds (fractions in most cases) for execution.

[Listing 2.2.3](#) lets you enter the number you want to sum up to (n). It then invokes the `sumOfN` method 25 times and calculates the time required for each trial:

Listing 2.2.3.

```

1  import java.util.Scanner;
2
3  public class Timing {
4      public static long sumOfN(long n) {
5          long theSum = 0;
6          for (int i = 1; i <= n; i++) {
7              theSum = theSum + i;
8          }
9          return theSum;
10     }
11
12     public static void main(String[] args) {
13
14         Scanner input = new Scanner(System.in);
15         System.out.print("Find sum from 1 to n: ");
16         long n = input.nextInt();
17
18         for (int trial = 0; trial < 25; trial++) {
19             long startTime = System.nanoTime();
20             long result = sumOfN(n);
21
22             double elapsed = (System.nanoTime() - startTime) / 1.0E9;
23             System.out.printf("Trial %d: Sum %d: time %.6f sec.%n",
24                             trial, result, elapsed);
25         }
26     }
27 }

```

[Listing 2.2.3](#) shows the original `sumOfN` function with the timing calls embedded before and after the summation. The function returns a tuple consisting of the result and the amount of time (in seconds) required for the calculation. Here is start and end of the output when we enter 10000 for the limit of the sum:

```

Trial 0: Sum 50005000: time 0.003886 sec.
Trial 1: Sum 50005000: time 0.004009 sec.
Trial 2: Sum 50005000: time 0.000186 sec.
Trial 3: Sum 50005000: time 0.000185 sec.
...
Trial 20: Sum 50005000: time 0.000125 sec.
Trial 21: Sum 50005000: time 0.000124 sec.
Trial 22: Sum 50005000: time 0.000125 sec.
Trial 23: Sum 50005000: time 0.000124 sec.
Trial 24: Sum 50005000: time 0.000124 sec.

```

Why does the time go down from .003886 seconds to .000124? Because the Java Virtual machine and the computer hardware itself **cache** results, keeping them in memory for future access. We run the trial loop 25 times in order to give the cache time to stabilize.

We discover that the time is fairly consistent and it takes on average about 0.000125 seconds to execute that code. What if we run the function adding the first 100,000 integers? (Showing only the final five trials)

```
Trial 20: Sum 5000050000: time 0.001225 sec.  
Trial 21: Sum 5000050000: time 0.001226 sec.  
Trial 22: Sum 5000050000: time 0.001225 sec.  
Trial 23: Sum 5000050000: time 0.001224 sec.  
Trial 24: Sum 5000050000: time 0.001224 sec.
```

Again, the time required for each run, although longer, is very consistent, averaging about 10 times more seconds. For n equal to 1,000,000 we get:

```
Trial 20: Sum 500000500000: time 0.012350 sec.  
Trial 21: Sum 500000500000: time 0.012411 sec.  
Trial 22: Sum 500000500000: time 0.012353 sec.  
Trial 23: Sum 500000500000: time 0.012443 sec.  
Trial 24: Sum 500000500000: time 0.012447 sec.
```

In this case, the average again turns out to be about 10 times the previous experiment.

Now consider [Listing 2.2.4](#), which shows a different means of solving the summation problem. This method, `sumOfNImproved`, takes advantage of a closed equation $\sum_{i=1}^n i = \frac{(n)(n+1)}{2}$ to compute the sum of the first n integers without iterating.

Listing 2.2.4.

```
public static long sumOfNImproved(long n) {  
    long theSum = n * (n + 1) / 2;  
    return theSum;  
}
```

We then change the call in line 23 of [Listing 2.2.3](#) to:

```
long result = sumOfNImproved(n);
```

If we do the same benchmark measurement with this revised code, using five different values for n (10,000, 100,000, 1,000,000, 10,000,000, and 100,000,000), we get the following results from averaging the last five trials:

```
Sum 50005000:      time 0.0000088 sec.  
Sum 5000050000:    time 0.0000092 sec.  
Sum 500000500000:  time 0.0000082 sec.  
Sum 5000000500000: time 0.0000078 sec.
```

There are two important things to notice about this output. First, the times recorded above are shorter than any of the previous examples. Second, they are very consistent no matter what the value of n . It appears that `sumOfNImproved` is hardly impacted by the number of integers being added.

But what does this benchmark really tell us? Intuitively, we can see that the iterative solutions seem to be doing more work since some program steps are being repeated. This is likely the reason it is taking longer. Also, the time required for the iterative solution seems to increase as we increase the value of n . However, if we ran the same function on a different computer or used a different method language, we would likely get different results. It could take even longer to perform `sumOfNImproved` if the computer were older.

We need a better way to characterize these algorithms with respect to execution time. The benchmark technique computes the actual time to execute. It does not really provide us with a useful measurement because it is dependent on a particular machine, program, time of day, compiler, and programming language. Instead, we would like to have a characterization that is independent of the program or computer being used. This measure would then be useful for judging the algorithm alone and could be used to compare algorithms across implementations.

You have attempted 1 of 1 activities on this page.



Building agents? Free virtual conference, Oct 22. 122 speakers, zero vendor pitches. **Register today**

www.alldayai.com

Ads by EthicalAds